

Microservices Resiliency

What is Resilience?

In distributed systems, **anything can happen**: network failures, service crashes, slow responses, or unexpected load spikes. (*Remember the [Fallacies of Distributed Systems](#)!*)

Resilience is the system's ability to:

- **Recover from failures**
- **Continue operating** despite partial failures
- **Prevent cascading failures** across services

An issue in one system can propagate and **affect the behavior and performance of others**.

Think of resilience as a combination of **fault-tolerance**, **graceful degradation**, and **self-healing**.

State of Resilience 2025 Survey

- **95%** aware of operational weaknesses
- **84%** lost $\geq \$10,000$ due to outages
- **55%** experienced weekly disruptions
- **39%** reported employee burnout from outages

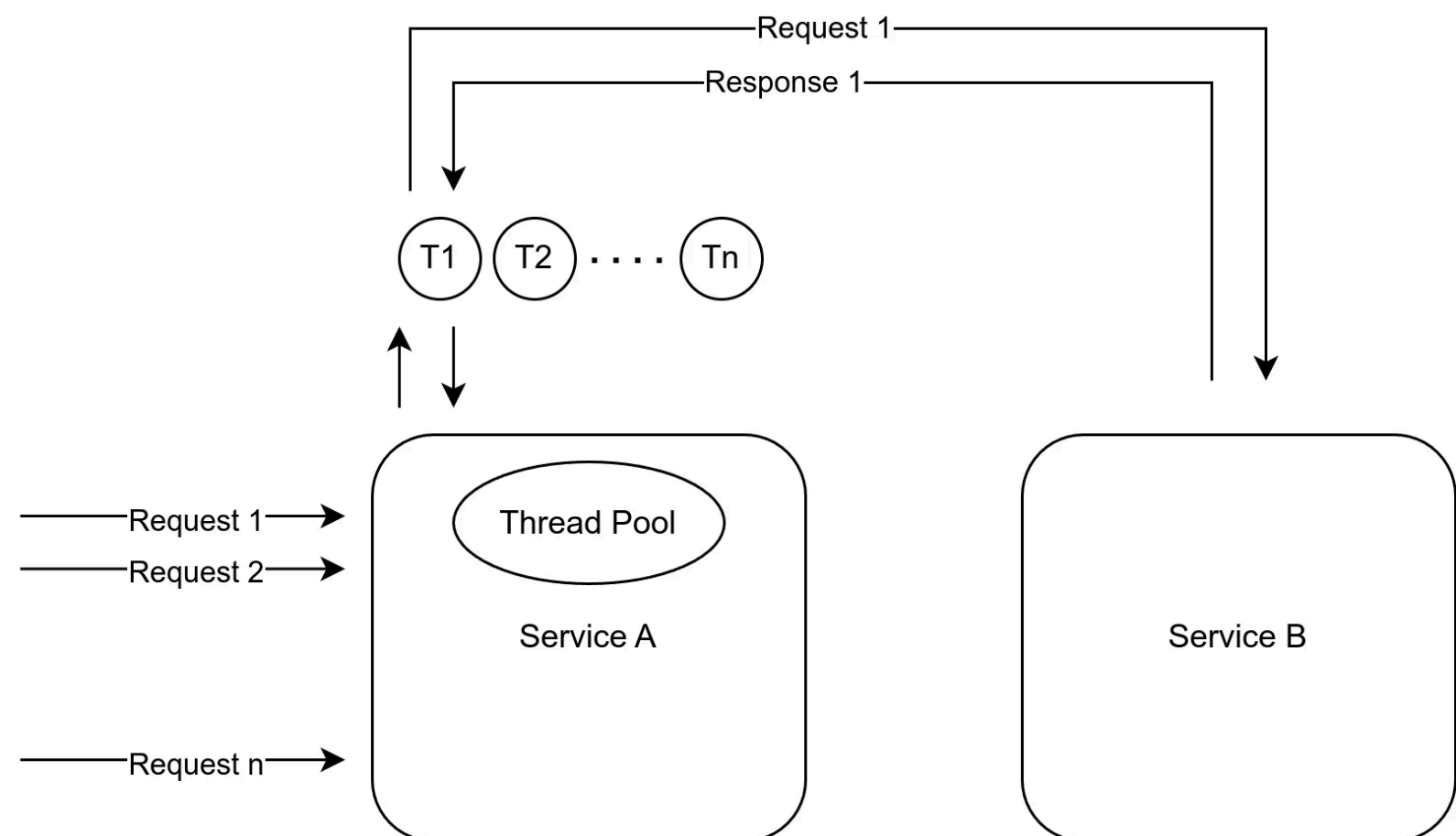
Outages are frequent, costly, and damaging to both business and staff morale.

Thread-Related Risks

One Thread Per Request

Traditional synchronous model:

- Service A assigns a **dedicated thread** to handle each request to Service B.
- The thread is **blocked while waiting**, consuming memory and some CPU. ([Spring Boot Starter Web](#)) enables this model.



Problems at scale:

1. **Memory consumption:** Each thread uses ~2MB; thousands of concurrent requests quickly consume gigabytes.

2. **Thread pool saturation:** Once the pool (`server.tomcat.threads.max`) is full, new requests queue → increased latency → potential rejection (DoS).

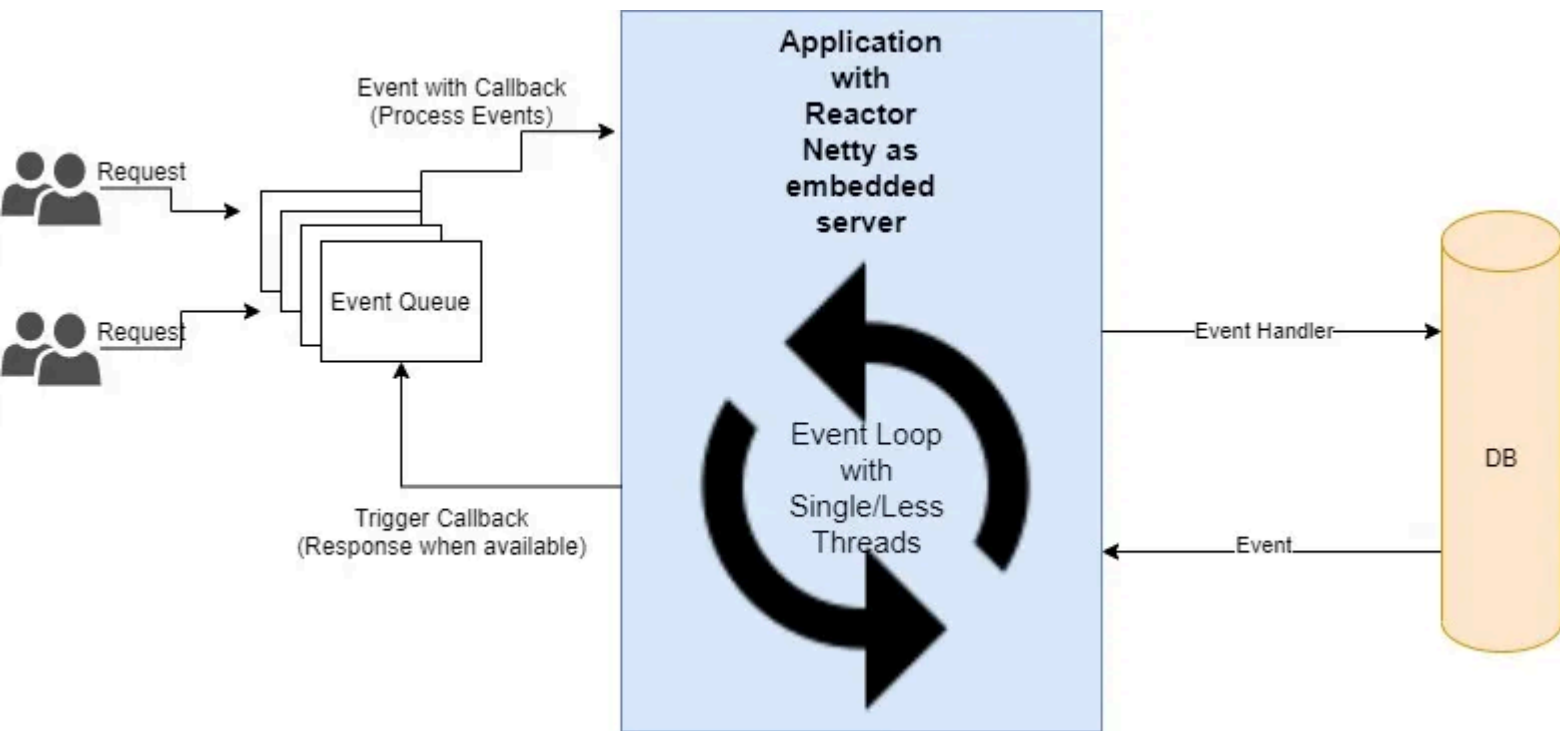
3. **Resource inefficiency:** Idle threads still consume CPU to manage scheduling, leaving less CPU for actual work.

*This model works for low-to-medium traffic but can **collapse under high load or slow downstream services**.*

Reactive Programming

Idea: Handle requests asynchronously with **non-blocking I/O**.

- Operations like DB queries or HTTP calls **don't block threads**.
- Instead, a **callback** resumes the operation once data is ready. ([Spring Boot WebFlux](#), [Quarkus](#), [FastAPI](#))



Key benefits:

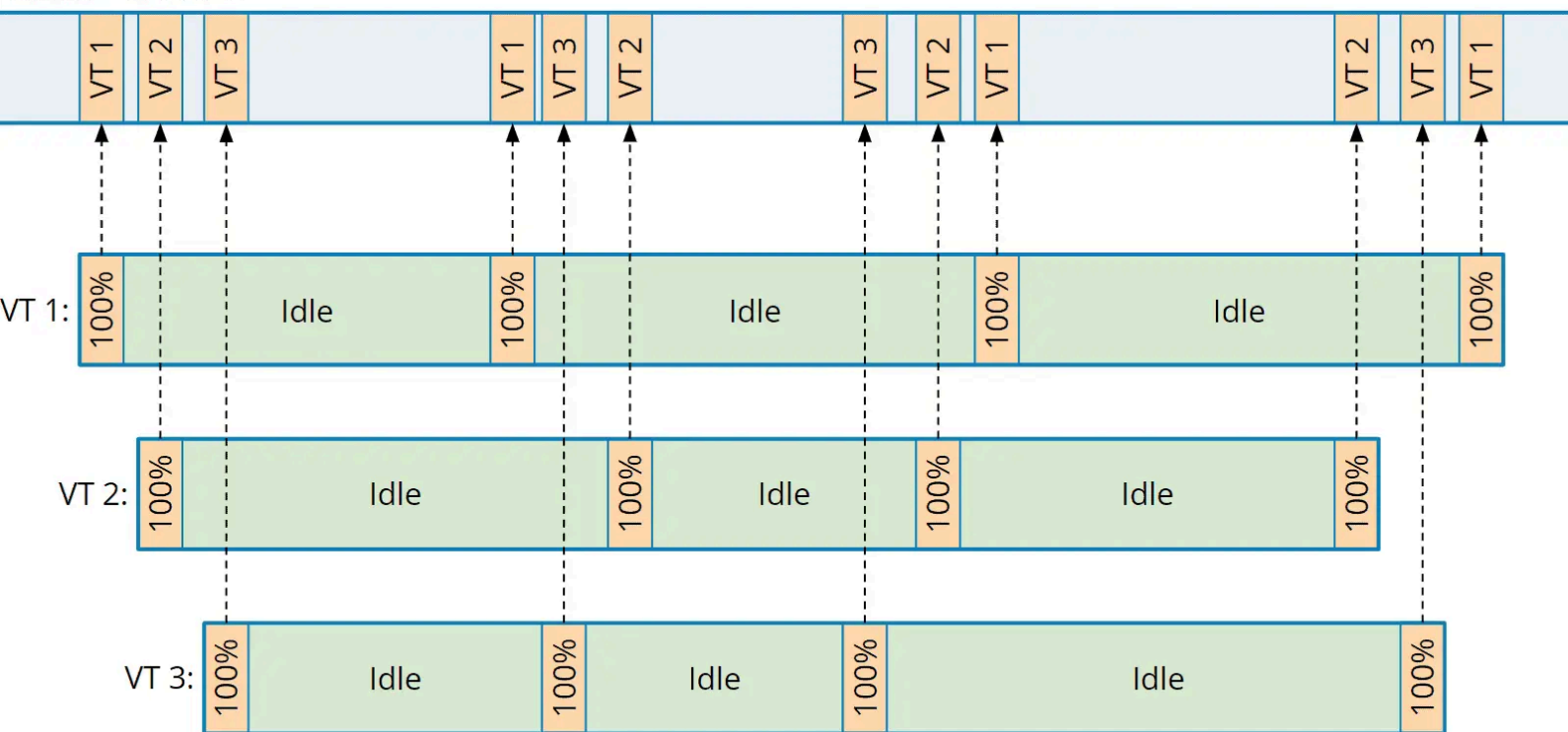
- **High concurrency** → can handle thousands of requests on a single thread
- **Better CPU utilization** → threads do actual computation instead of waiting (co-routines as used for waiting)

Java Virtual Threads

A modern alternative to both thread-per-request and reactive models.

- Lightweight threads managed by the **JVM**, not the OS.
- Traditional **blocking code can be used**, but at massive scale.

Carrier Thread 1:



Key features:

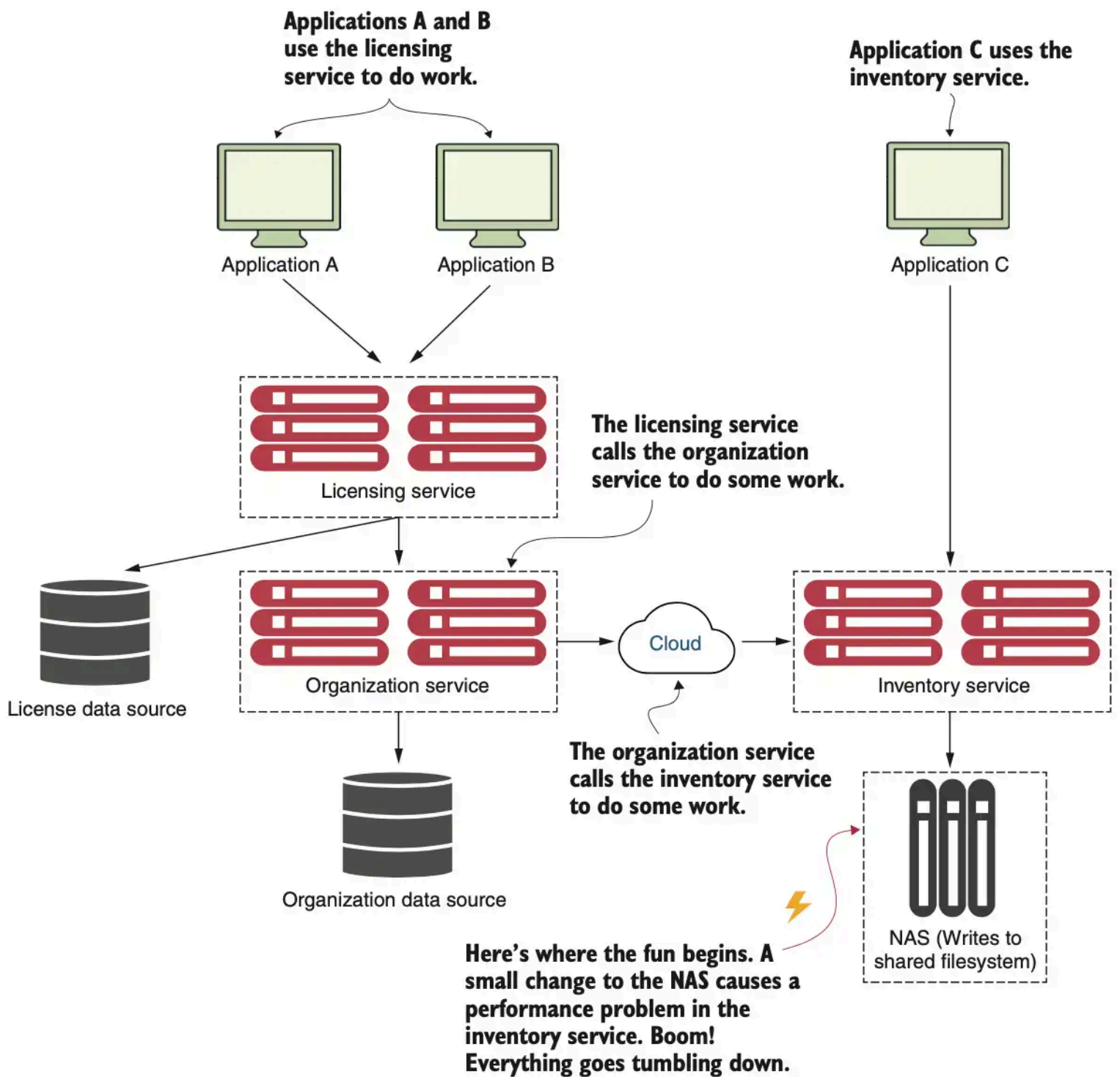
- **Cheap to create and manage:** thousands of virtual threads possible with minimal overhead.
- **Simplifies code:** no callback chains or reactive frameworks needed.
- **Compatible with existing libraries:** works with blocking I/O libraries without rewriting code.

Best for teams who want **simplicity + high concurrency**, without learning reactive paradigms.

Real-World example

A minor NAS configuration change caused:

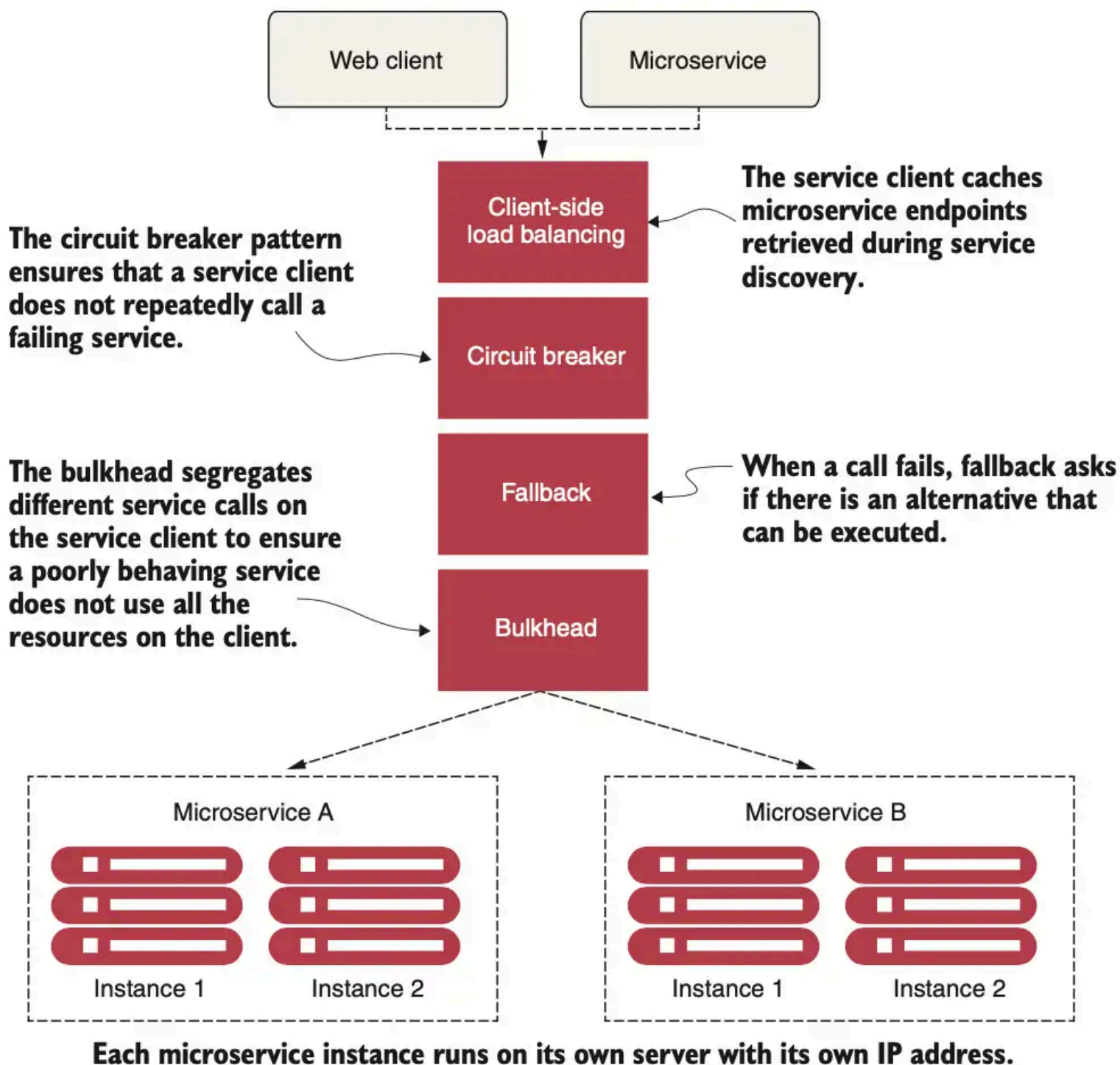
1. Inventory service reads slowed dramatically
2. Organization service threads and DB connections were **exhausted**
3. Licensing service calls queued → **cascading failure across three services**



Single slow service → ripple effect → entire application ecosystem impacted.

Client-Side Resiliency Patterns

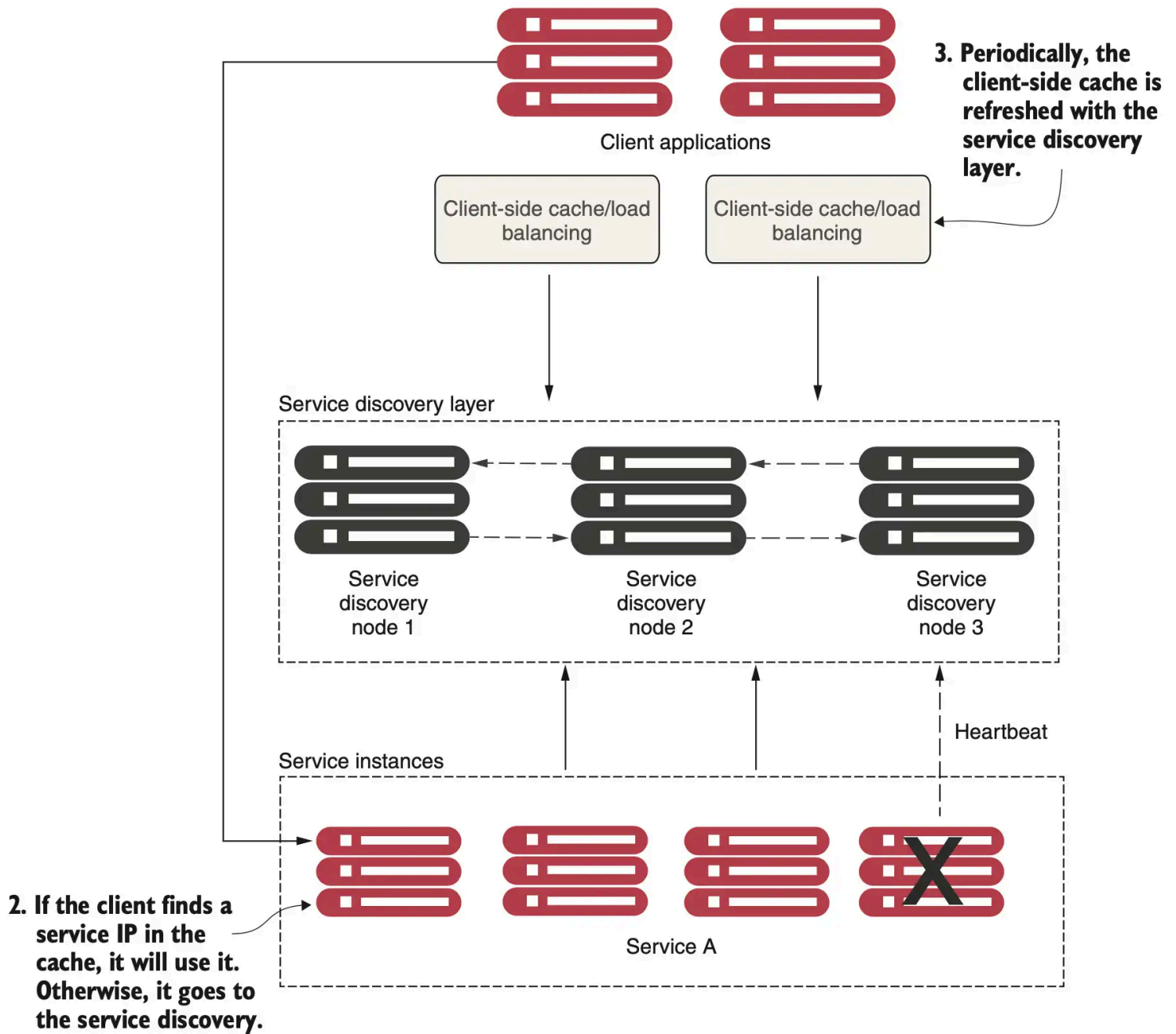
Protect **clients** from remote service failures.



Load Balancing

- Client maintains **cache of service instances** (via Eureka or similar).
- Removes failing instances dynamically.

1. When a service client needs to call a service, it checks a local cache for the service instance IPs. Load balancing between service instances occurs on the service.

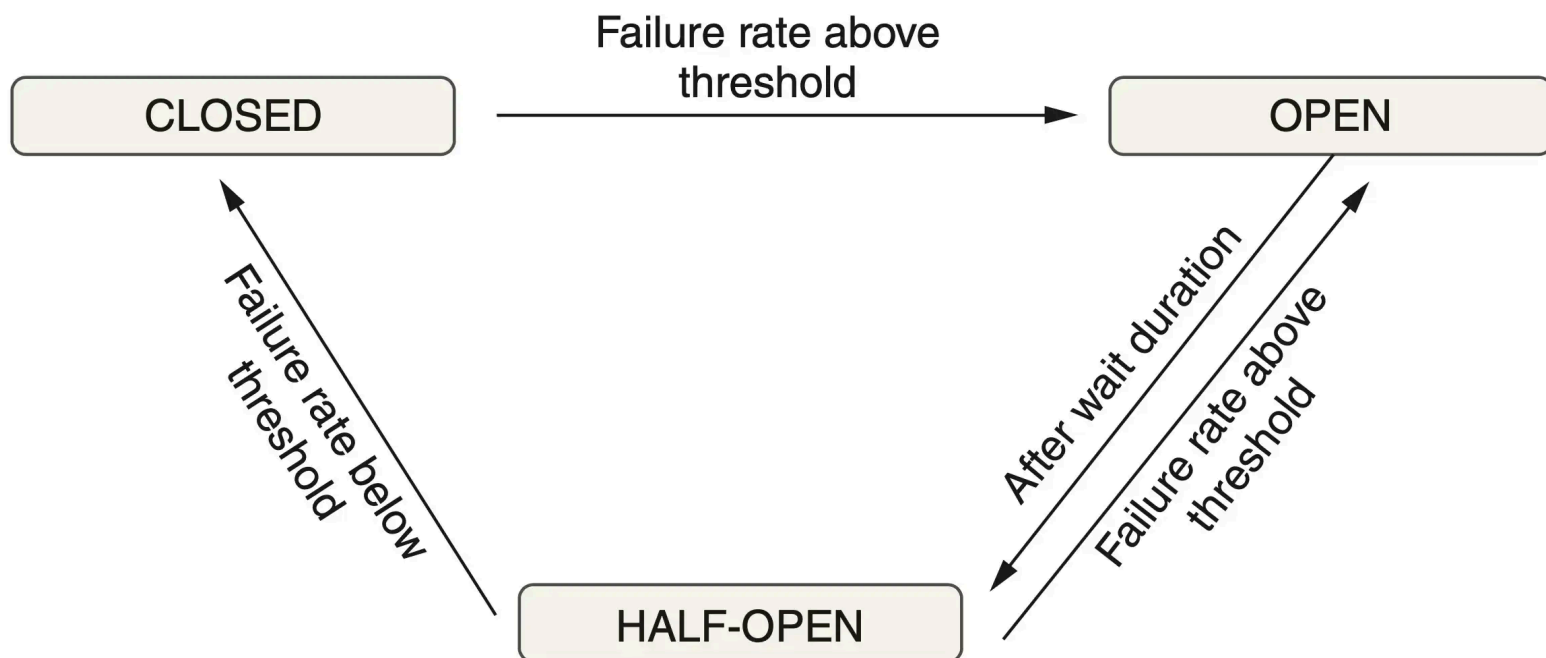


Benefit: Prevents requests from hitting unhealthy services, reducing cascading failures.

Circuit Breaker

Monitors dependent services and **fails fast** if unhealthy.

- **Open:** block calls, use fallback
- **Half-open:** test if service recovered
- **Closed:** normal operations



Fixed Window

- Count failures in fixed intervals (e.g., 10s).
- Open circuit if threshold exceeded.
- **Pros:** Simple. **Cons:** Slow to detect bursts at window edges.

Sliding Window

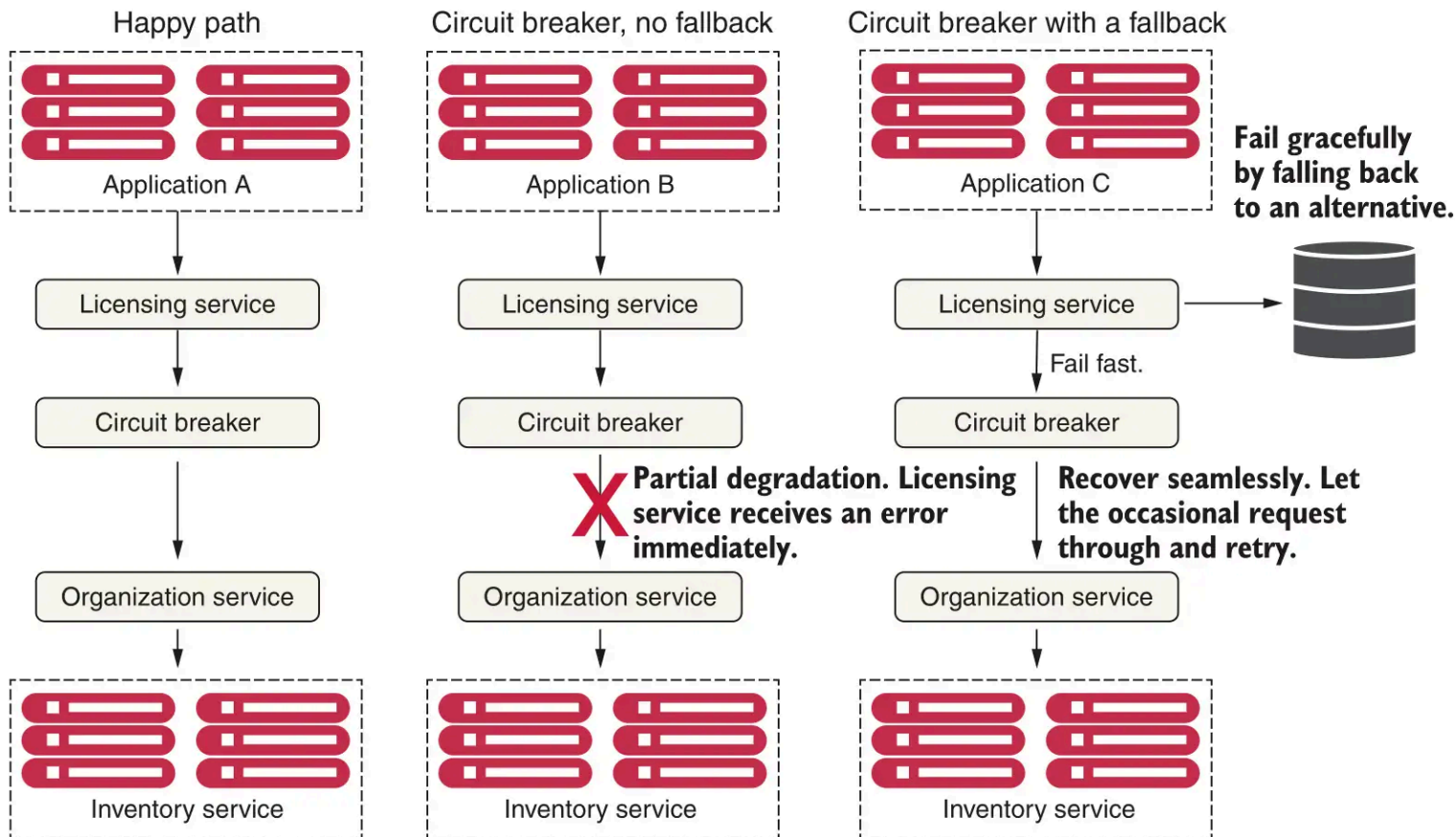
- Continuously track failures over a rolling period.
- Open circuit if failure ratio exceeds threshold.
- **Pros:** Fast detection. **Cons:** Sensitive to short bursts.

Leaky Bucket

- Failures accumulate in a "bucket" that leaks over time.
- Open circuit if bucket fills up.
- **Pros:** Smooths bursts. **Cons:** Needs tuning (bucket size/leak rate).

Fallback

- Execute alternative logic when remote calls fail
- Prevents **exceptions from crashing clients**



Example: Return cached data or default values if a service is down.

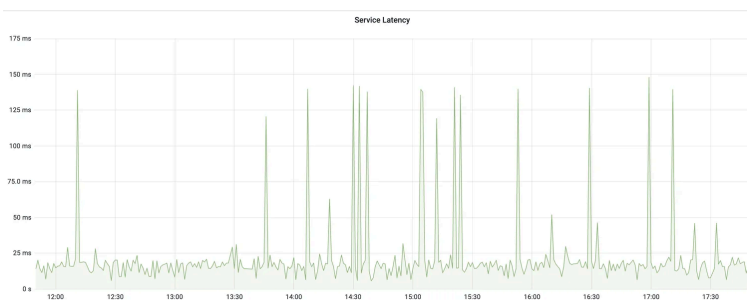
Retry

- Only for **idempotent operations**
- **Exponential backoff + jitter** smooths traffic spikes

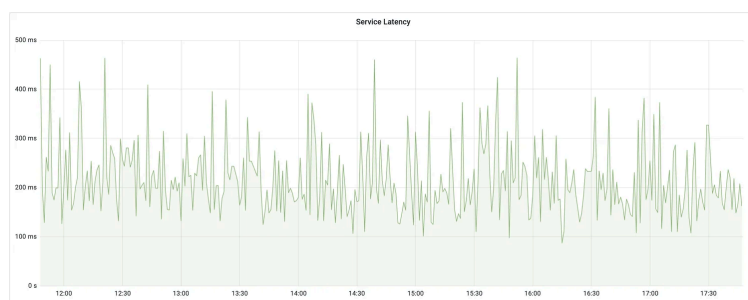
⚠ Network saturation risk: Use **circuit breakers** to prevent **retry storms**

- Retries increase traffic → more congestion
- More congestion → more failures
- More failures → even more retries

Retry suitable

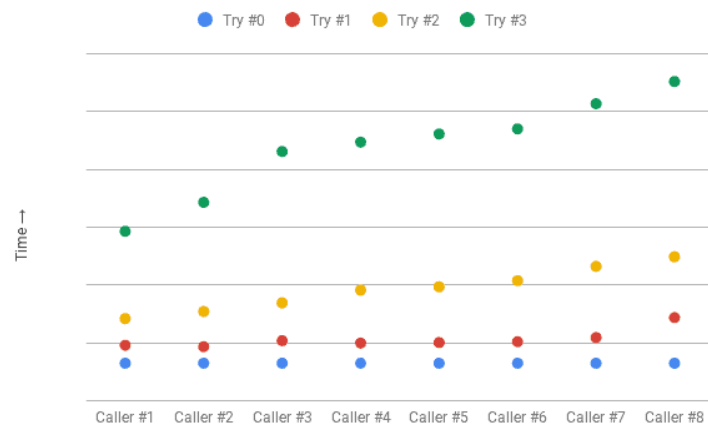
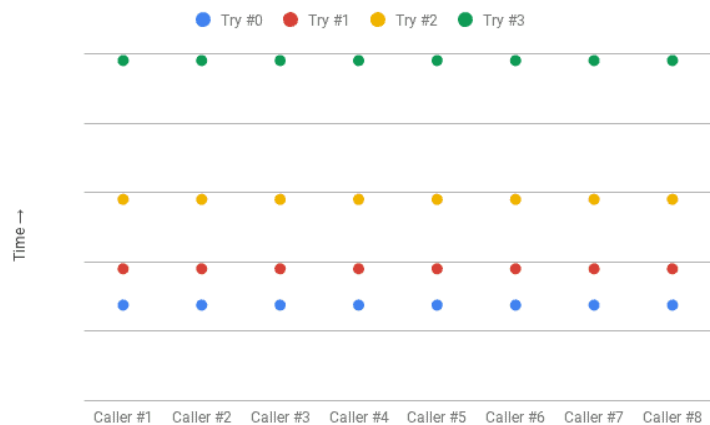


Retry not suitable



Backoff

Backoff + jitter



Tip: Avoid retries for **time-critical operations**; failing fast (**fallback**) is often better.

Timeout

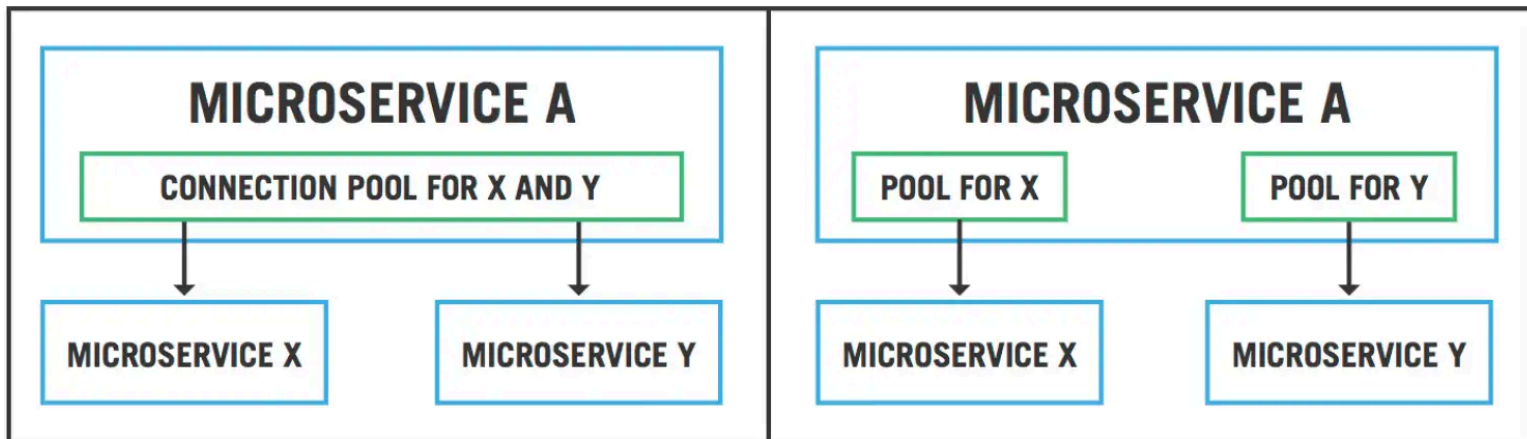
- Set maximum wait for synchronous calls
- **Too high:** wastes resources
- **Too low:** triggers unnecessary retries → latency spikes

Best practice: use **high-percentile latency metrics** (e.g., 99.9th percentile) to choose timeouts.

Bulkhead

Isolate resources per dependency to prevent one slow service from consuming all resources.

- Different thread pools or connection pools per downstream service
- Failures confined → other parts remain operational



Example: Service A with `/a/x` (slow) and `/a/y` (fast).

Without bulkheads → `/a/x` consumes all threads → `/a/y` blocked.

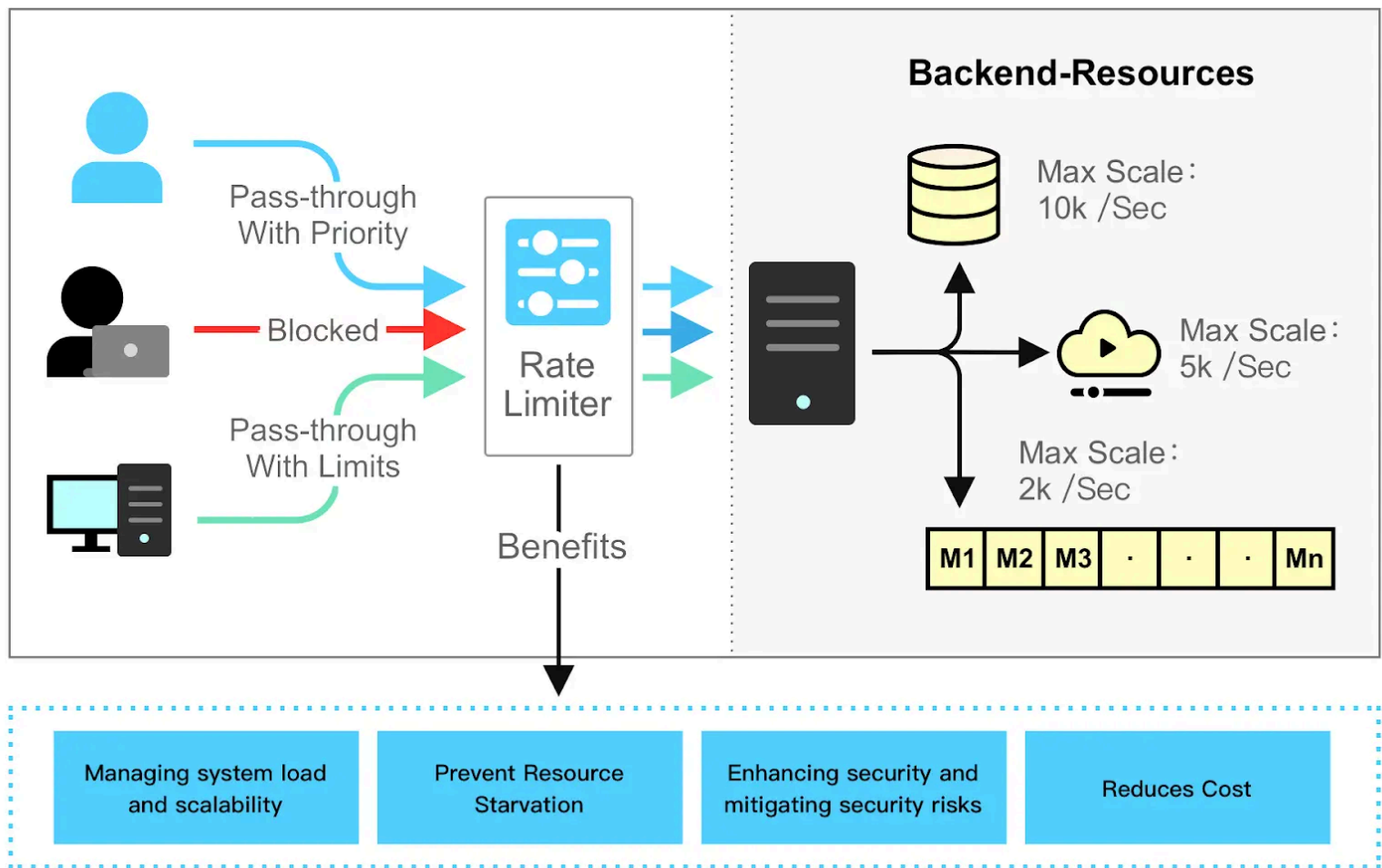
With bulkheads → `/a/x` threads isolated → `/a/y` continues.

Server-Side Resiliency Patterns

Rate Limiter

Controls the **rate of incoming requests**.

- Prevents overload, DoS attacks, and unfair resource usage
- Strategies: fixed window, sliding window



Load Shedding vs Rate Limiting:

- **Rate Limiting:** throttle **before processing**
- **Load Shedding:** drop requests **when overloaded**

Resources

- *Microservices with Spring Boot 3 and Spring Cloud* - Chapter 13
- [YouTube Video on Microservices Resiliency](#)
- AWS Article: [Exponential Backoff and Jitter](#)